



university of
groningen

Models and Semantics of Computation (MaSC)

Lecture 8: HOFL: Syntax, Types, Eager/Lazy Semantics

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands

Previously on MaSC



- ▶ IMP: A core language for imperative programming; syntax, op. semantics, rule induction.
- ▶ Lambda-notation.
- ▶ Today we look at HOFL: a core language for typed functional programming.

From IMP to HOFL



- ▶ IMP targets imperative programming with three fixed types / sorts.
- ▶ In HOFL, we have higher-order functional programs, with a variety of types, which have their own dedicated syntax.
- ▶ We use types to distinguish between pre-terms and terms.
- ▶ The type system is simple but sufficient to formally study properties of functional programs.
- ▶ Two evaluation strategies for programs: lazy and eager.

HOFL: Syntax


$$\begin{array}{l} t ::= x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ \mid \lambda x. t \mid t_0 \ t_1 \\ \mid \text{rec } x. t \end{array}$$

ordinary constructs

pairs, projections

abstraction, application

recursion

HOFL: Syntax


$$t ::= x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1$$
$$\mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t)$$
$$\mid \lambda x. t \mid t_0 \ t_1$$
$$\mid \text{rec } x. t$$

ordinary constructs

pairs, projections

abstraction, application

recursion

HOFL: Syntax



No Bool! Read: **if** $t = 0$ **then** t_0 **else** t_1

$t ::= x \mid n \mid t_0 \text{ op } t_1 \mid$
 $(t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t)$
 $\lambda x. t \mid t_0 \ t_1$
 rec $x. t$

ordinary constructs

pairs, projections

abstraction, application

recursion

Guess the Meaning



► `rec f. λx. if x then 1 else x × (f (x - 1))`

Guess the Meaning



► `rec f. λx. if x then 1 else x × (f (x - 1))`

(factorial)

Guess the Meaning



► `rec f. λx. if x then 1 else x × (f (x - 1))`

(factorial)

► `rec rep. λn. λf. λx. if n then x else f(rep(n - 1) f x)`

Guess the Meaning



- ▶ $\text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f (x - 1))$ (factorial)
- ▶ $\text{rec } \text{rep}. \lambda n. \lambda f. \lambda x. \text{if } n \text{ then } x \text{ else } f(\text{rep}(n - 1) f x)$ ($\text{rep } n f x = f^n x$)

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

► $x + 1$

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 \ t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

► $x + 1$



Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 \ t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$ ✓

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$ ✓
- ▶ $1 + (0, 5)$

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$ ✓
- ▶ $1 + (0, 5)$ ✗

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 \ t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$ ✓
- ▶ $1 + (0, 5)$ ✗
- ▶ **fst**(3)

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 \ t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$ ✓
- ▶ $1 + (0, 5)$ ✗
- ▶ **fst**(3) ✗

Pre-terms



We call **pre-terms** the terms generated by the syntax:

$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 \ t_1 \\ & \mid \text{rec } x. t \end{aligned}$$

We have **well-formed** pre-terms, but also **ill-formed** ones:

- ▶ $x + 1$ ✓
- ▶ **if** x **then** $x + 1$ **else** $x - 1$ ✓
- ▶ $1 + (0, 5)$ ✗
- ▶ **fst**(3) ✗

We need a **type system**!



Type System

Syntax of Types



We write $\mathcal{T}$ to denote the set of all types:

$$\tau ::= \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1$$

Syntax of Types



We write $\mathcal{T}$ to denote the set of all types:

Integers (base type)

$\tau ::= \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1$

Syntax of Types



We write $\mathcal{T}$ to denote the set of all types:

Integers (base type)

Pair type (aka product, related to conjunction)

$\tau ::= \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1$

Syntax of Types



We write $\mathcal{T}$ to denote the set of all types:

Integers (base type)

Pair type (aka product, related to conjunction)

$$\tau ::= \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1$$

Function type (aka arrow, related to implication)

Syntax of Types



$$\tau ::= \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1$$

Example

`int * int`
`int * (int → int)`
`(int * int) → int`

a pair of integers
a pair that includes a function
function from pairs to integers

Syntax of Types



$$\tau ::= \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1$$

Example

<code>int * int</code>	a pair of integers
<code>int * (int → int)</code>	a pair that includes a function
<code>(int * int) → int</code>	function from pairs to integers

Relating variables/identifiers to types:

- ▶ We assume variables are typed: $\text{Ide} = \{\text{Ide}_\tau\}_{\tau \in \mathcal{T}}$.
- ▶ There is a function $(\hat{\cdot}) : \text{Ide} \rightarrow \mathcal{T}$. This way, $\hat{x}$ denotes the type of x .
- ▶ **Typing judgments** with formulas `$t : \tau$` (read: t has type τ)
- ▶ Types are assigned to pre-terms using a set of inference rules

$$\begin{array}{c} \frac{}{x : \hat{x}} \quad \frac{}{n : \text{int}} \quad \frac{t_0 : \text{int} \quad t_1 : \text{int}}{t_0 \text{ op } t_1 : \text{int}} \quad \frac{t : \text{int} \quad t_0 : \tau \quad t_1 : \tau}{\text{if } t \text{ then } t_0 \text{ else } t_1 : \tau} \\[10pt] \frac{t_0 : \tau_0 \quad t_1 : \tau_1}{(t_0, t_1) : \tau_0 * \tau_1} \quad \frac{t : \tau_0 * \tau_1}{\text{fst}(t) : \tau_0} \quad \frac{t : \tau_0 * \tau_1}{\text{snd}(t) : \tau_1} \quad \frac{x : \tau_0 \quad t : \tau_1}{\lambda x. t : \tau_0 \rightarrow \tau_1} \\[10pt] \frac{t_1 : \tau_0 \rightarrow \tau_1 \quad t_0 : \tau_0}{t_1 t_0 : \tau_1} \quad \frac{x : \tau \quad t : \tau}{\text{rec } x. t : \tau} \end{array}$$

Well-formed Terms



$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \\ \tau ::= & \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1 \end{aligned}$$

- A pre-term t is **well-formed** (well-typed, typable) if $\exists \tau \in \mathcal{T}. t : \tau$.
That is, t is well-formed if we can assign a type to t using the rules.

Well-formed Terms



$$\begin{aligned} t ::= & x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \\ & \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \\ & \mid \lambda x. t \mid t_0 t_1 \\ & \mid \text{rec } x. t \\ \tau ::= & \text{int} \mid \tau_0 * \tau_1 \mid \tau_0 \rightarrow \tau_1 \end{aligned}$$

- ▶ A pre-term t is **well-formed** (well-typed, typable) if $\exists \tau \in \mathcal{T}. t : \tau$.
That is, t is well-formed if we can assign a type to t using the rules.
- ▶ We only give semantics to well-formed pre-terms.
- ▶ The set of all well-formed terms of type τ is denoted T_τ .

The Type System is Simple



Consider the example:

$$t \triangleq \text{rec } p. \lambda x. (x, p(x + 2))$$

- Intuitively, this is a sequence of all even numbers

$$t\ 0 \equiv (0, (t\ 2)) \equiv (0, (2, (t\ 4))) \equiv \dots \equiv (0, (2, (4, \dots)))$$

The Type System is Simple



Consider the example:

$$t \triangleq \text{rec } p. \lambda x. (x, p(x + 2))$$

- Intuitively, this is a sequence of all even numbers

$$t\ 0 \equiv (0, (t\ 2)) \equiv (0, (2, (t\ 4))) \equiv \dots \equiv (0, (2, (4, \dots)))$$

- Note: the type system is simple enough to type sequences of integers of finite length, but we have no type for sequences of arbitrary/infinite length.

The Type System is Simple



Consider the example:

$$t \triangleq \text{rec } p. \lambda x. (x, p(x + 2))$$

- Intuitively, this is a sequence of all even numbers

$$t\ 0 \equiv (0, (t\ 2)) \equiv (0, (2, (t\ 4))) \equiv \dots \equiv (0, (2, (4, \dots)))$$

- Note: the type system is simple enough to type sequences of integers of finite length, but we have no type for sequences of arbitrary/infinite length.
- (A more powerful tool: **recursive types**.)

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$fact : \mathbf{int} \rightarrow \mathbf{int}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}}$$

$$fact : \mathbf{int} \rightarrow \mathbf{int}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}}{fact : \mathbf{int} \rightarrow \mathbf{int}}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}}}{\lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}} \quad fact : \mathbf{int} \rightarrow \mathbf{int}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{}{\mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int}}}{\lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}} \quad fact : \mathbf{int} \rightarrow \mathbf{int}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \frac{\hat{x} = \mathbf{int} \quad \frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}}}{\mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int}}}{\lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}}}{fact : \mathbf{int} \rightarrow \mathbf{int}}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \ x \ \mathbf{then} \ 1 \ \mathbf{else} \ (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\frac{\hat{f} = \text{int} \rightarrow \text{int}}{f : \text{int} \rightarrow \text{int}} \quad \frac{\frac{\hat{x} = \text{int}}{x : \text{int}} \quad \frac{\hat{x} = \text{int}}{1 : \text{int}}}{\text{if } x \text{ then } 1 \text{ else } (x \times (f(x - 1))) : \text{int}}}{\lambda x : \text{int}. \text{if } x \text{ then } 1 \text{ else } (x \times (f(x - 1))) : \text{int} \rightarrow \text{int}}}{fact : \text{int} \rightarrow \text{int}}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{1 : \mathbf{int}}{1 : \mathbf{int}} \quad \frac{(x \times (f(x - 1))) : \mathbf{int}}{(x \times (f(x - 1))) : \mathbf{int}}}{\frac{\mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int}}{\lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}}} \quad fact : \mathbf{int} \rightarrow \mathbf{int}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad 1 : \mathbf{int}}{(x \times (f(x - 1))) : \mathbf{int}}}{\mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int}}}{\lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}}}{fact : \mathbf{int} \rightarrow \mathbf{int}}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**. Example:

$$fact \triangleq \mathbf{rec} \, f : \mathbf{int} \rightarrow \mathbf{int}. \lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1)))$$

Using the rules:

$$\frac{\frac{\frac{\hat{f} = \mathbf{int} \rightarrow \mathbf{int}}{f : \mathbf{int} \rightarrow \mathbf{int}} \quad \frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{\frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad \frac{\frac{\hat{x} = \mathbf{int}}{x : \mathbf{int}} \quad 1 : \mathbf{int}}{(x \times (f(x - 1))) : \mathbf{int}}}{\mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int}}}{\lambda x : \mathbf{int}. \mathbf{if} \, x \, \mathbf{then} \, 1 \, \mathbf{else} \, (x \times (f(x - 1))) : \mathbf{int} \rightarrow \mathbf{int}}}{fact : \mathbf{int} \rightarrow \mathbf{int}}$$

Church-style Typability ($\sim$ Type Checking)



Variables are tagged with (declared) types.

We deduce the type of terms by **structural induction**.

Alternatively:

$$\begin{array}{c} fact \triangleq \text{rec } \underset{\text{int} \rightarrow \text{int}}{f} . \lambda \underset{\text{int}}{x} . \text{if } \underset{\text{int}}{x} \text{ then } \underset{\text{int}}{1} \text{ else } (\underset{\text{int}}{x} \times (\underset{\text{int} \rightarrow \text{int}}{f} (\underset{\text{int}}{x} - \underset{\text{int}}{1}))) \\ \hspace{15em} \underbrace{\hspace{10em}}_{\text{int}} \\ \hspace{10em} \underbrace{\hspace{10em}}_{\text{int}} \\ \hspace{5em} \underbrace{\hspace{10em}}_{\text{int}} \\ \underbrace{\hspace{10em}}_{\text{int}} \\ \underbrace{\hspace{10em}}_{\text{int} \rightarrow \text{int}} \end{array}$$

Curry-style Typability ($\sim$ Type Inference)



- ▶ The types of variables are not given. Rather, type rules are used to derive **type constraints** (type equations).
- ▶ Solutions of those equations (via unification, using a signature induced by the syntax of types) define so-called **principal types**.

Curry-style Typability ($\sim$ Type Inference)



- ▶ The types of variables are not given. Rather, type rules are used to derive **type constraints** (type equations).
- ▶ Solutions of those equations (via unification, using a signature induced by the syntax of types) define so-called **principal types**.

Example

Consider the identity function $\lambda x. x$. We have:

$$\begin{array}{c} \lambda x. x : \tau \quad \nwarrow \tau = \tau_1 \rightarrow \tau_2, \hat{x} = \tau_1 \quad x : \tau_2 \\ \nwarrow \hat{x} = \tau_2 \quad \square \end{array}$$

We have $\hat{x} = \tau_1 = \tau_2$. The principal type is $\tau_1 \rightarrow \tau_1$, for **any arbitrary** τ_1 .

Curry-style Typability: Example



Recall:

$$t \triangleq \text{rec } p. \lambda x. (x, p(x + 2))$$

Concisely:

$$t \triangleq \text{rec } \underbrace{p}_{\text{int} \rightarrow \tau_4} . \lambda \underbrace{x}_{\text{int}} . \left(\underbrace{x}_{\text{int}}, \underbrace{p}_{\text{int} \rightarrow \tau_4} \left(\underbrace{x}_{\text{int}} + \underbrace{2}_{\text{int}} \right) \right)$$

$\underbrace{\hspace{10em}}_{\text{int}}$
 $\underbrace{\hspace{10em}}_{\tau_4}$
 $\underbrace{\hspace{10em}}_{\text{int} * \tau_4}$
 $\underbrace{\hspace{10em}}_{\text{int} \rightarrow (\text{int} * \tau_4)}$

Curry-style Typability: Example



Recall:

$$t \triangleq \text{rec } p. \lambda x. (x, p(x + 2))$$

Concisely:

$$t \triangleq \text{rec } \underbrace{p}_{\text{int} \rightarrow \tau_4} . \lambda \underbrace{x}_{\text{int}} . \left(\underbrace{x}_{\text{int}}, \underbrace{p}_{\text{int} \rightarrow \tau_4} \left(\underbrace{x}_{\text{int}} + \underbrace{2}_{\text{int}} \right) \right)$$

$\underbrace{\hspace{15em}}_{\text{int}}$
 $\underbrace{\hspace{10em}}_{\tau_4}$
 $\underbrace{\hspace{10em}}_{\text{int} * \tau_4}$
 $\underbrace{\hspace{15em}}_{\text{int} \rightarrow (\text{int} * \tau_4)}$

But then $\text{int} \rightarrow \tau_4 \stackrel{?}{=} \text{int} \rightarrow (\text{int} * \tau_4)$ implies

$$\tau_4 \stackrel{?}{=} (\text{int} * \tau_4)$$

Hence, unification fails (occur-check)!

Curry-style Typability: Example



rec $p. \lambda x. (x, (p(x + 2))) : \tau$

Curry-style Typability: Example



$$\mathbf{rec} \, p. \lambda x. (x, (p(x + 2))) : \tau \preceq_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\preceq_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\preceq_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\nwarrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\nwarrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\nwarrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\nwarrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\nwarrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\nwarrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \\ &\nwarrow_{\hat{x}=\tau_3} (p(x + 2)) : \tau_4 \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\nwarrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\nwarrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\nwarrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \\ &\nwarrow_{\hat{x}=\tau_3} (p(x + 2)) : \tau_4 \\ &\nwarrow p : \tau_5 \rightarrow \tau_4, (x + 2) : \tau_5 \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\nwarrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\nwarrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\nwarrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \\ &\nwarrow_{\hat{x}=\tau_3} (p(x + 2)) : \tau_4 \\ &\nwarrow_{\phantom{\hat{x}=\tau_3}} p : \tau_5 \rightarrow \tau_4, (x + 2) : \tau_5 \\ &\nwarrow_{\hat{p}=\tau_5 \rightarrow \tau_4} (x + 2) : \tau_5 \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\nwarrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\nwarrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\nwarrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \\ &\nwarrow_{\hat{x}=\tau_3} (p(x + 2)) : \tau_4 \\ &\nwarrow_{\hat{p}=\tau_5 \rightarrow \tau_4} p : \tau_5 \rightarrow \tau_4, (x + 2) : \tau_5 \\ &\nwarrow_{\tau_5=\text{int}} x : \text{int}, 2 : \text{int} \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\nwarrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\nwarrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\nwarrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \\ &\nwarrow_{\hat{x}=\tau_3} (p(x + 2)) : \tau_4 \\ &\nwarrow p : \tau_5 \rightarrow \tau_4, (x + 2) : \tau_5 \\ &\nwarrow_{\hat{p}=\tau_5 \rightarrow \tau_4} (x + 2) : \tau_5 \\ &\nwarrow_{\tau_5=\text{int}} x : \text{int}, 2 : \text{int} \\ &\nwarrow_{\hat{x}=\text{int}}^* \square \end{aligned}$$

Curry-style Typability: Example



$$\begin{aligned} \text{rec } p. \lambda x. (x, (p(x + 2))) : \tau &\searrow_{\hat{p}=\tau} \lambda x. (x, p(x + 2)) : \tau \\ &\searrow_{\tau=\tau_1 \rightarrow \tau_2, \hat{x}=\tau_1} (x, (p(x + 2))) : \tau_2 \\ &\searrow_{\tau_2=\tau_3 * \tau_4} x : \tau_3, (p(x + 2)) : \tau_4 \\ &\searrow_{\hat{x}=\tau_3} (p(x + 2)) : \tau_4 \\ &\searrow_{\hat{p}=\tau_5 \rightarrow \tau_4} p : \tau_5 \rightarrow \tau_4, (x + 2) : \tau_5 \\ &\searrow_{\tau_5=\text{int}} x : \text{int}, 2 : \text{int} \\ &\searrow_{\hat{x}=\text{int}}^* \square \end{aligned}$$

We have derived:

- ▶ $\hat{x} = \tau_1, \hat{x} = \tau_3, \hat{x} = \text{int}$. Therefore: $\tau_1 = \tau_3 = \text{int}$
- ▶ $\hat{p} = \tau = \tau_1 \rightarrow \tau_2, \hat{p} = \tau_5 \rightarrow \tau_4$. Therefore, $\tau_1 = \tau_5 = \text{int}$ and $\tau_2 = \tau_4$.
- ▶ We have both $\tau_2 = \tau_4$ and $\tau_2 = \tau_3 * \tau_4$. Therefore, we derive fail.



Semantics



$$\text{fv}(n) \triangleq \emptyset$$

$$\text{fv}(x) \triangleq \{x\}$$

$$\text{fv}(t_0 \text{ op } t_1) \triangleq \text{fv}(t_0) \cup \text{fv}(t_1)$$

$$\text{fv}(\text{if } t \text{ then } t_0 \text{ else } t_1) \triangleq \text{fv}(t) \cup \text{fv}(t_0) \cup \text{fv}(t_1)$$

$$\text{fv}((t_0, t_1)) \triangleq \text{fv}(t_0) \cup \text{fv}(t_1)$$

$$\text{fv}(\text{fst}(t)) \triangleq \text{fv}(t)$$

$$\text{fv}(\text{snd}(t)) \triangleq \text{fv}(t)$$

$$\text{fv}(\lambda x. t) \triangleq \text{fv}(t) \setminus \{x\}$$

$$\text{fv}((t_0 \ t_1)) \triangleq \text{fv}(t_0) \cup \text{fv}(t_1)$$

$$\text{fv}(\text{rec } x. t) \triangleq \text{fv}(t) \setminus \{x\}$$

We say t is **closed** if $\text{fv}(t) = \emptyset$.

Capture-Avoiding Substitution



$$n[t/x] \triangleq n$$

$$y[t/x] \triangleq \text{if } y = x \text{ then } t \text{ else } y$$

$$(t_0 \text{ op } t_1)[t/x] \triangleq t_0[t/x] \text{ op } t_1[t/x]$$

$$(\text{if } t' \text{ then } t_0 \text{ else } t_1)[t/x] \triangleq \text{if } t'[t/x] \text{ then } t_0[t/x] \text{ else } t_1[t/x]$$

$$(t_0, t_1)[t/x] \triangleq (t_0[t/x], t_1[t/x])$$

$$\text{fst}(t')[t/x] \triangleq \text{fst}(t'[t/x])$$

$$\text{snd}(t')[t/x] \triangleq \text{snd}(t'[t/x])$$

$$(t_0 \ t_1)[t/x] \triangleq (t_0[t/x] \ t_1[t/x])$$

$$(\lambda y. t')[t/x] \triangleq \lambda z. ((t'[z/y])[t/x])$$

$$\text{if } z \notin \text{fv}(\lambda y. e') \cup \text{fv}(e) \cup \{x\}$$

$$(\text{rec } x. t')[t/x] \triangleq \text{rec } z. (t'[z/y])[t/x]$$

$$\text{if } z \notin \text{fv}(\text{rec } y. t') \cup \text{fv}(e) \cup \{x\}$$

Substitution Respects Types



Theorem

Given $x_0 : \tau_0$, $t_0 : \tau_0$, and $t : \tau$, then $t[t_0/x_0] : \tau$

The proof is by

Substitution Respects Types



Theorem

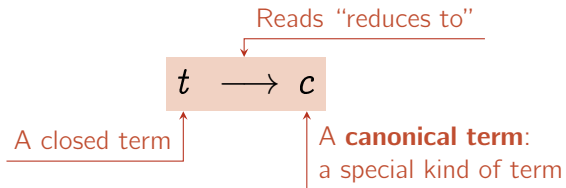
Given $x_0 : \tau_0$, $t_0 : \tau_0$, and $t : \tau$, then $t[t_0/x_0] : \tau$

The proof is by **structural induction** on the stronger assertion $t[\tilde{t}/\tilde{x}] : \tau$.

Big-Step Operational Semantics



Formalizes the computation of **canonical forms** by term manipulation:





The set of canonical forms of type τ is denoted $C_\tau \subseteq T_\tau$.

$$\frac{}{n \in C_{\text{int}}}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}}$$

t_0, t_1 may not be in canonical form: **laziness**

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

t not necessarily a closed term

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}}$$

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$
 $(1, 2)$
 $(1 + 1, 2 - 1)$
 $\text{fst}(1, 2)$

$\text{if } 0 \text{ then } 0 \text{ else } 0$
 $\lambda x. 1$
 $\lambda x. 1 + 2 \times 3$
 $\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}}$$

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ **X**

$(1, 2)$

$(1 + 1, 2 - 1)$

fst(1, 2)

if 0 **then** 0 **else** 0

$\lambda x. 1$

$\lambda x. 1 + 2 \times 3$

$\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}} \quad \frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}} \quad \frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ ✗
 $(1, 2)$
 $(1 + 1, 2 - 1)$
 fst $(1, 2)$

if 0 **then** 0 **else** 0 ✗
 $\lambda x. 1$
 $\lambda x. 1 + 2 \times 3$
 $\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}} \quad \frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}} \quad \frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ ✗

$(1, 2)$ ✓

$(1 + 1, 2 - 1)$

fst(1, 2)

if 0 **then** 0 **else** 0 ✗

$\lambda x. 1$

$\lambda x. 1 + 2 \times 3$

$\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}}$$

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ ✗

$(1, 2)$ ✓

$(1 + 1, 2 - 1)$

fst(1, 2)

if 0 **then** 0 **else** 0 ✗

$\lambda x. 1$ ✓

$\lambda x. 1 + 2 \times 3$

$\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}} \quad \frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}} \quad \frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ ✗
 $(1, 2)$ ✓
 $(1 + 1, 2 - 1)$ ✓
 $\text{fst}(1, 2)$

$\text{if } 0 \text{ then } 0 \text{ else } 0$ ✗
 $\lambda x. 1$ ✓
 $\lambda x. 1 + 2 \times 3$ ✓
 $\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}}$$

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ ✗

$(1, 2)$ ✓

$(1 + 1, 2 - 1)$ ✓

fst(1, 2) ✗

if 0 **then** 0 **else** 0 ✗

$\lambda x. 1$ ✓

$\lambda x. 1 + 2 \times 3$ ✓

$\lambda x. \text{fst}(1, 2)$

Example: Canonical Forms?



$$\frac{}{n \in C_{\text{int}}}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \in C_{\tau_0 * \tau_1}}$$

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \in C_{\tau_0 \rightarrow \tau_1}}$$

$1 + 2 \times 3$ ✗

$(1, 2)$ ✓

$(1 + 1, 2 - 1)$ ✓

fst(1, 2) ✗

if 0 **then** 0 **else** 0 ✗

$\lambda x. 1$ ✓

$\lambda x. 1 + 2 \times 3$ ✓

$\lambda x. \text{fst}(1, 2)$ ✓

Operational Semantics: Axioms



A single rule:

$$\frac{c \in C_{\tau}}{c \longrightarrow c}$$

Expanding the several cases we have:

Operational Semantics: Axioms



A single rule:

$$\frac{c \in C_\tau}{c \longrightarrow c}$$

Expanding the several cases we have:

$$\frac{}{n \longrightarrow n}$$

$$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \longrightarrow (t_0, t_1)}$$

$$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \longrightarrow \lambda x. t}$$

That is: integers, pairs, and abstractions are already in canonical form.

$\frac{}{n \longrightarrow n}$	$\frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \longrightarrow (t_0, t_1)}$	$\frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \longrightarrow \lambda x. t}$
$\frac{t_0 \longrightarrow n_0 \quad t_1 \longrightarrow n_1}{t_0 \text{ op } t_1 \longrightarrow n_0 \text{ op } n_1}$	$\frac{t \longrightarrow 0 \quad t_0 \longrightarrow c_0}{\text{if } t \text{ then } t_0 \text{ else } t_1 \longrightarrow c_0}$	$\frac{t \longrightarrow (t_0, t_1) \quad t_0 \longrightarrow c_0}{\text{fst}(t) \longrightarrow c_0}$
$\frac{t[\text{rec } x. t/x] \longrightarrow c}{\text{rec } x. t \longrightarrow c}$	$\frac{t \longrightarrow n \quad n \neq 0 \quad t_1 \longrightarrow c_1}{\text{if } t \text{ then } t_0 \text{ else } t_1 \longrightarrow c_1}$	$\frac{t \longrightarrow (t_0, t_1) \quad t_1 \longrightarrow c_1}{\text{snd}(t) \longrightarrow c_1}$

Lazy Semantics



$$\begin{array}{c}
 \frac{}{n \longrightarrow n} \qquad \frac{t_0 : \tau_0 \quad t_1 : \tau_1 \quad t_0, t_1 \text{ closed}}{(t_0, t_1) \longrightarrow (t_0, t_1)} \qquad \frac{\lambda x. t : \tau_0 \rightarrow \tau_1 \quad \lambda x. t \text{ closed}}{\lambda x. t \longrightarrow \lambda x. t} \\
 \\
 \frac{t_0 \longrightarrow n_0 \quad t_1 \longrightarrow n_1}{t_0 \text{ op } t_1 \longrightarrow n_0 \text{ op } n_1} \qquad \frac{t \longrightarrow 0 \quad t_0 \longrightarrow c_0}{\text{if } t \text{ then } t_0 \text{ else } t_1 \longrightarrow c_0} \qquad \frac{t \longrightarrow (t_0, t_1) \quad t_0 \longrightarrow c_0}{\text{fst}(t) \longrightarrow c_0} \\
 \\
 \frac{t[\text{rec } x. t/x] \longrightarrow c}{\text{rec } x. t \longrightarrow c} \qquad \frac{t \longrightarrow n \quad n \neq 0 \quad t_1 \longrightarrow c_1}{\text{if } t \text{ then } t_0 \text{ else } t_1 \longrightarrow c_1} \qquad \frac{t \longrightarrow (t_0, t_1) \quad t_1 \longrightarrow c_1}{\text{snd}(t) \longrightarrow c_1} \\
 \\
 \text{(lazy)} \\
 \frac{t_1 \longrightarrow \lambda x. t'_1 \quad t'_1[t_0/x] \longrightarrow c}{(t_1 \ t_0) \longrightarrow c}
 \end{array}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c?$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$t(1, 2) \longrightarrow c \quad \nwarrow \quad t \longrightarrow \lambda x'. t', \quad t'[(1, 2)/x'] \longrightarrow c$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\swarrow t \longrightarrow \lambda x'. t', t'[(1, 2)/x'] \longrightarrow c \\ &\swarrow_{x'=x, t'=\text{if} \dots} (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\quad \nwarrow \quad t \longrightarrow \lambda x'. t', \quad t'[(1, 2)/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots} \quad (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &\quad = \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\quad \nwarrow \quad t \longrightarrow \lambda x'. t', \quad t'[(1, 2)/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots} \quad (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &\quad = \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \\ &\quad \nwarrow \quad \text{fst}(1, 2) \longrightarrow n, \quad n \neq 0, \quad \text{snd}(1, 2) \longrightarrow c \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\quad \nwarrow \quad t \longrightarrow \lambda x'. t', \quad t'[(1, 2)/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots} \quad (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &\quad = \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \\ &\quad \nwarrow \quad \text{fst}(1, 2) \longrightarrow n, \quad n \neq 0, \quad \text{snd}(1, 2) \longrightarrow c \\ &\quad \nwarrow \quad (1, 2) \longrightarrow (n_1, n_2), \quad n_1 \longrightarrow n, \quad n \neq 0, \quad \text{snd}(1, 2) \longrightarrow c \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\swarrow t \longrightarrow \lambda x'. t', t'[(1, 2)/x'] \longrightarrow c \\ &\swarrow_{x'=x, t'=\text{if} \dots} (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &= \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \\ &\swarrow \text{fst}(1, 2) \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\swarrow (1, 2) \longrightarrow (n_1, n_2), n_1 \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\swarrow_{n_1=1, n_2=2, n=1}^* \text{snd}(1, 2) \longrightarrow c \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\searrow t \longrightarrow \lambda x'. t', t'[(1, 2)/x'] \longrightarrow c \\ &\searrow_{x'=x, t'=\text{if} \dots} (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &= \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \\ &\searrow \text{fst}(1, 2) \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\searrow (1, 2) \longrightarrow (n_1, n_2), n_1 \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\searrow^*_{n_1=1, n_2=2, n=1} \text{snd}(1, 2) \longrightarrow c \\ &\searrow (1, 2) \longrightarrow (n_3, n_4), n_4 \longrightarrow c \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\swarrow t \longrightarrow \lambda x'. t', t'[(1, 2)/x'] \longrightarrow c \\ &\swarrow_{x'=x, t'=\text{if} \dots} (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &= \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \\ &\swarrow \text{fst}(1, 2) \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\swarrow (1, 2) \longrightarrow (n_1, n_2), n_1 \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\swarrow_{n_1=1, n_2=2, n=1}^* \text{snd}(1, 2) \longrightarrow c \\ &\swarrow (1, 2) \longrightarrow (n_3, n_4), n_4 \longrightarrow c \\ &\swarrow_{n_3=1, n_4=2, c=2}^* \square \end{aligned}$$

Example



Let $t \triangleq \lambda x. \text{if fst}(x) \text{ then } 1 \text{ else snd}(x) : (\text{int} * \text{int}) \rightarrow \text{int}.$

What is $t(1, 2) \longrightarrow c$?

$$\begin{aligned} t(1, 2) \longrightarrow c &\swarrow t \longrightarrow \lambda x'. t', t'[(1, 2)/x'] \longrightarrow c \\ &\swarrow_{x'=x, t'=\text{if} \dots} (\text{if fst}(x) \text{ then } 1 \text{ else snd}(x))[(1, 2)/x] \longrightarrow c \\ &= \text{if fst}(1, 2) \text{ then } 1 \text{ else snd}(1, 2) \longrightarrow c \\ &\swarrow \text{fst}(1, 2) \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\swarrow (1, 2) \longrightarrow (n_1, n_2), n_1 \longrightarrow n, n \neq 0, \text{snd}(1, 2) \longrightarrow c \\ &\swarrow_{n_1=1, n_2=2, n=1}^* \text{snd}(1, 2) \longrightarrow c \\ &\swarrow (1, 2) \longrightarrow (n_3, n_4), n_4 \longrightarrow c \\ &\swarrow_{n_3=1, n_4=2, c=2}^* \square \end{aligned} \quad \text{Hence, } \boxed{t(1, 2) \longrightarrow 2}$$

Example



Let t be defined as follows:

$$t \triangleq \text{rec } \underbrace{x . x}_{\tau} : \tau$$

We have:

$$\begin{aligned} \text{rec } x . x &\longrightarrow c \nwarrow x[\text{rec } x . x / x] \longrightarrow c \\ &= \text{rec } x . x \longrightarrow c \end{aligned}$$

We reach the same goal from which we started, with no other option to explore:
divergence!

Example

Let $fact \triangleq \text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f(x - 1))$. We have:



Example

Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$(fact\ 1) \longrightarrow c \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c$$



Example



Let $fact \triangleq \text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots}^* \quad (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1/x] \longrightarrow c \end{aligned}$$

Example



Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad = \mathbf{if} 1 \mathbf{then} 1 \mathbf{else} 1 \times (fact(1 - 1)) \longrightarrow c \end{aligned}$$

Example



Let $fact \triangleq \text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots}^* (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad \quad = \text{if } 1 \text{ then } 1 \text{ else } 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow \quad 1 \longrightarrow n, \quad n \neq 0, \quad 1 \times (fact(1 - 1)) \longrightarrow c \end{aligned}$$

Example



Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \mathbf{if} 1 \mathbf{then} 1 \mathbf{else} 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow \quad 1 \longrightarrow n, \quad n \neq 0, \quad 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow_{n=1, c=n_1 \times n_2}^* \quad 1 \longrightarrow n_1, \quad (fact(1 - 1)) \longrightarrow n_2 \end{aligned}$$

Example



Let $fact \triangleq \text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots}^* (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \text{if } 1 \text{ then } 1 \text{ else } 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow \quad 1 \longrightarrow n, \quad n \neq 0, \quad 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, \quad (fact(1 - 1)) \longrightarrow n_2 \\ &\quad \nwarrow_{n=1} \quad fact \longrightarrow \lambda x''. t'', \quad t'' [1 - 1/x''] \longrightarrow n_2 \end{aligned}$$

Example



Let $fact \triangleq \text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots}^* (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \text{if } 1 \text{ then } 1 \text{ else } 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow \quad 1 \longrightarrow n, \quad n \neq 0, \quad 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, \quad (fact(1 - 1)) \longrightarrow n_2 \\ &\quad \nwarrow_{n=1} \quad fact \longrightarrow \lambda x''. t'', \quad t''[1 - 1/x''] \longrightarrow n_2 \\ &\quad \nwarrow_{x''=x, t''=\text{if} \dots}^* (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1 - 1/x] \longrightarrow n_2 \end{aligned}$$

Example



Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \mathbf{if} 1 \mathbf{then} 1 \mathbf{else} 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow \quad 1 \longrightarrow n, \quad n \neq 0, \quad 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, \quad (fact(1 - 1)) \longrightarrow n_2 \\ &\quad \nwarrow_{n=1} \quad fact \longrightarrow \lambda x''. t'', \quad t''[1 - 1/x''] \longrightarrow n_2 \\ &\quad \nwarrow_{x''=x, t''=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1 - 1/x] \longrightarrow n_2 \\ &\quad \quad = \mathbf{if} 1 - 1 \mathbf{then} 1 \mathbf{else} (1 - 1) \times (fact((1 - 1) - 1)) \longrightarrow n_2 \end{aligned}$$

Example



Let $fact \triangleq \text{rec } f. \lambda x. \text{if } x \text{ then } 1 \text{ else } x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', \quad t'[1/x'] \longrightarrow c \\ &\quad \nwarrow_{x'=x, t'=\text{if} \dots}^* (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \text{if } 1 \text{ then } 1 \text{ else } 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow \quad 1 \longrightarrow n, \quad n \neq 0, \quad 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \nwarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, \quad (fact(1 - 1)) \longrightarrow n_2 \\ &\quad \nwarrow_{n=1} fact \longrightarrow \lambda x''. t'', \quad t''[1 - 1/x''] \longrightarrow n_2 \\ &\quad \nwarrow_{x''=x, t''=\text{if} \dots}^* (\text{if } x \text{ then } 1 \text{ else } x \times (fact(x - 1)))[1 - 1/x] \longrightarrow n_2 \\ &\quad \quad = \text{if } 1 - 1 \text{ then } 1 \text{ else } (1 - 1) \times (fact((1 - 1) - 1)) \longrightarrow n_2 \\ &\quad \nwarrow \quad 1 - 1 \longrightarrow 0, \quad 1 \longrightarrow n_2 \end{aligned}$$

Example



Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \swarrow \quad fact \longrightarrow \lambda x'. t', t'[1/x'] \longrightarrow c \\ &\quad \swarrow_{x'=x, t'=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \mathbf{if} 1 \mathbf{then} 1 \mathbf{else} 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \swarrow \quad 1 \longrightarrow n, n \neq 0, 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \swarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, (fact(1 - 1)) \longrightarrow n_2 \\ &\quad \swarrow_{n=1} \quad fact \longrightarrow \lambda x''. t'', t''[1 - 1/x''] \longrightarrow n_2 \\ &\quad \swarrow_{x''=x, t''=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1 - 1/x] \longrightarrow n_2 \\ &\quad \quad = \mathbf{if} 1 - 1 \mathbf{then} 1 \mathbf{else} (1 - 1) \times (fact((1 - 1) - 1)) \longrightarrow n_2 \\ &\quad \swarrow \quad 1 - 1 \longrightarrow 0, 1 \longrightarrow n_2 \\ &\quad \swarrow_{n_2=1}^* \quad \square \end{aligned}$$

Example



Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$\begin{aligned} (fact\ 1) &\longrightarrow c \quad \swarrow \quad fact \longrightarrow \lambda x'. t', t'[1/x'] \longrightarrow c \\ &\quad \swarrow_{x'=x, t'=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1/x] \longrightarrow c \\ &\quad \quad = \mathbf{if} 1 \mathbf{then} 1 \mathbf{else} 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \swarrow \quad 1 \longrightarrow n, n \neq 0, 1 \times (fact(1 - 1)) \longrightarrow c \\ &\quad \swarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, (fact(1 - 1)) \longrightarrow n_2 \\ &\quad \swarrow_{n=1} fact \longrightarrow \lambda x''. t'', t''[1 - 1/x''] \longrightarrow n_2 \\ &\quad \swarrow_{x''=x, t''=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1 - 1/x] \longrightarrow n_2 \\ &\quad \quad = \mathbf{if} 1 - 1 \mathbf{then} 1 \mathbf{else} (1 - 1) \times (fact((1 - 1) - 1)) \longrightarrow n_2 \\ &\quad \swarrow \quad 1 - 1 \longrightarrow 0, 1 \longrightarrow n_2 \\ &\quad \swarrow_{n_2=1}^* \square \quad \quad \quad \text{Hence, } \boxed{c = n_1 \times n_2 = 1 \times 1 = 1.} \end{aligned}$$

Example



Let $fact \triangleq \mathbf{rec} f. \lambda x. \mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (f(x - 1))$. We have:

$$(fact\ 1) \longrightarrow c \quad \nwarrow \quad fact \longrightarrow \lambda x'. t', t'[1/x'] \longrightarrow c$$

$$\begin{aligned} & \nwarrow_{x'=x, t'=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1/x] \longrightarrow c \\ & \quad = \mathbf{if} 1 \mathbf{then} 1 \mathbf{else} 1 \times (fact(1 - 1)) \longrightarrow c \end{aligned}$$

$$\nwarrow \quad 1 \longrightarrow n, n \neq 0, 1 \times (fact(1 - 1)) \longrightarrow c$$

$$\nwarrow_{n=1, c=n_1 \times n_2}^* 1 \longrightarrow n_1, (fact(1 - 1)) \longrightarrow n_2 \quad \text{Laziness evident here!}$$

$$\nwarrow_{n=1} fact \longrightarrow \lambda x''. t'', t''[1 - 1/x''] \longrightarrow n_2$$

$$\begin{aligned} & \nwarrow_{x''=x, t''=\mathbf{if} \dots}^* (\mathbf{if} x \mathbf{then} 1 \mathbf{else} x \times (fact(x - 1)))[1 - 1/x] \longrightarrow n_2 \\ & \quad = \mathbf{if} 1 - 1 \mathbf{then} 1 \mathbf{else} (1 - 1) \times (fact((1 - 1) - 1)) \longrightarrow n_2 \end{aligned}$$

$$\nwarrow \quad 1 - 1 \longrightarrow 0, 1 \longrightarrow n_2$$

$$\nwarrow_{n_2=1}^* \square \quad \text{Hence, } \boxed{c = n_1 \times n_2 = 1 \times 1 = 1.}$$

Properties of the Semantics



► Termination? $\forall t. \exists c. t \longrightarrow c?$ ✗

Counterexample: **rec** $x. x$.

Properties of the Semantics



- Termination? $\forall t. \exists c. t \longrightarrow c$? ✗

Counterexample: **rec** $x. x$.

- Determinacy? $\forall t. \forall c_1, c_2. t \longrightarrow c_1 \wedge t \longrightarrow c_2 \Rightarrow c_1 = c_2$? ✓

Prove $P(t \longrightarrow c) \triangleq \forall c_1. t \longrightarrow c_1 \Rightarrow c_1 = c$ by rule induction.

Properties of the Semantics



- ▶ **Termination?** $\forall t. \exists c. t \longrightarrow c$? ✗
Counterexample: **rec** $x. x$.
- ▶ **Determinacy?** $\forall t. \forall c_1, c_2. t \longrightarrow c_1 \wedge t \longrightarrow c_2 \Rightarrow c_1 = c_2$? ✓
Prove $P(t \longrightarrow c) \triangleq \forall c_1. t \longrightarrow c_1 \Rightarrow c_1 = c$ by **rule induction**.
- ▶ **Subject reduction?** $\forall t. \forall c. \forall \tau. t \longrightarrow c \wedge t : \tau \Rightarrow c : \tau$? ✓
Prove $P(t \longrightarrow c) \triangleq \forall \tau. t : \tau \Rightarrow c : \tau$ by **rule induction**.

Properties of the Semantics



- ▶ **Termination?** $\forall t. \exists c. t \longrightarrow c$? ✗
Counterexample: **rec** $x. x$.
- ▶ **Determinacy?** $\forall t. \forall c_1, c_2. t \longrightarrow c_1 \wedge t \longrightarrow c_2 \Rightarrow c_1 = c_2$? ✓
Prove $P(t \longrightarrow c) \triangleq \forall c_1. t \longrightarrow c_1 \Rightarrow c_1 = c$ by **rule induction**.
- ▶ **Subject reduction?** $\forall t. \forall c. \forall \tau. t \longrightarrow c \wedge t : \tau \Rightarrow c : \tau$? ✓
Prove $P(t \longrightarrow c) \triangleq \forall \tau. t : \tau \Rightarrow c : \tau$ by **rule induction**.
- ▶ **Congruence?** Given $t_1 \equiv_{\text{op}} t_2$ iff $\forall c. (t_1 \longrightarrow c \Leftrightarrow t_2 \longrightarrow c)$.
Is it a congruence? ✗
Counterexample: $2 \equiv_{\text{op}} 1 + 1$ but $\lambda x. 2 \not\equiv_{\text{op}} \lambda x. 1 + 1$.

Lazy vs Eager Semantics



We have seen the rule

$$\begin{array}{c} \text{(lazy)} \\ t_1 \longrightarrow \lambda x. t'_1 \quad t'_1[t_0/x] \longrightarrow c \\ \hline (t_1 t_0) \longrightarrow c \end{array}$$

What would be an **eager** rule?

Lazy vs Eager Semantics



We have seen the rule

$$\begin{array}{c} \text{(lazy)} \\ t_1 \longrightarrow \lambda x. t'_1 \quad t'_1[t_0/x] \longrightarrow c \\ \hline (t_1 t_0) \longrightarrow c \end{array}$$

What would be an **eager** rule?

$$\begin{array}{c} \text{(eager)} \\ t_1 \longrightarrow \lambda x. t'_1 \quad t_0 \longrightarrow c_0 \quad t'_1[c_0/x] \longrightarrow c \\ \hline (t_1 t_0) \longrightarrow c \end{array}$$

Lazy vs Eager Semantics, Compared (1)


$$t \triangleq (\lambda x. 1)(\mathbf{rec} \ y. y) : \mathbf{int}$$

Lazy vs Eager Semantics, Compared (1)



$$t \triangleq (\lambda x. 1)(\text{rec } y. y) : \text{int}$$

Lazy

$$\begin{aligned} t &\longrightarrow c \quad \swarrow \quad \lambda x. 1 \longrightarrow \lambda x'. t', t'[\text{rec } y. y/x'] \longrightarrow c \\ &\quad \swarrow_{x'=x, t'=1} 1[\text{rec } y. y/x] \longrightarrow c \\ &\quad = 1 \longrightarrow c \\ &\quad \swarrow_{c=1} \square \end{aligned}$$

Lazy vs Eager Semantics, Compared (1)



$$t \triangleq (\lambda x. 1)(\text{rec } y. y) : \text{int}$$

Lazy

$$\begin{aligned} t \longrightarrow c &\searrow \lambda x. 1 \longrightarrow \lambda x'. t', t'[\text{rec } y. y/x'] \longrightarrow c \\ &\searrow_{x'=x, t'=1} 1[\text{rec } y. y/x] \longrightarrow c \\ &= 1 \longrightarrow c \\ &\searrow_{c=1} \square \end{aligned}$$

Eager

$$\begin{aligned} t \longrightarrow c &\searrow \lambda x. 1 \longrightarrow \lambda x'. t', \text{rec } y. y \longrightarrow c', t'[c'/x'] \longrightarrow c \\ &\searrow_{x'=x, t'=1} \text{rec } y. y \longrightarrow c', 1[c'/x'] \longrightarrow c \\ &\searrow \text{rec } y. y \longrightarrow c', 1[c'/x'] \longrightarrow c \end{aligned}$$

divergence!

Lazy vs Eager Semantics, Compared (2)


$$t \triangleq (\lambda x. x + x)(1 \times 2) : \text{int}$$

Lazy vs Eager Semantics, Compared (2)



$$t \triangleq (\lambda x. x + x)(1 \times 2) : \text{int}$$

Lazy

$$t \longrightarrow c \quad \nwarrow \quad \lambda x. x + x \longrightarrow \lambda x'. t', \quad t'[1 \times 2/x'] \longrightarrow c$$

$$\nwarrow \quad x'=x, t'=x+x \quad (x + x)[1 \times 2/x] \longrightarrow c$$

$$= (1 \times 2) + (1 \times 2) \longrightarrow c$$

$$\nwarrow \quad c=c_1 \pm c_2 \quad (1 \times 2) \longrightarrow c_1, \quad (1 \times 2) \longrightarrow c_2$$

$$\nwarrow \quad c=c_1 \pm c_2 \quad \square \quad c = c_1 \pm c_2 = 2 \pm 2 = 4$$

Lazy vs Eager Semantics, Compared (2)



$$t \triangleq (\lambda x. x + x)(1 \times 2) : \text{int}$$

Eager

$$\begin{aligned} t &\longrightarrow c \swarrow \lambda x. x + x \longrightarrow \lambda x'. t', 1 \times 2 \longrightarrow c', t'[c'/x'] \longrightarrow c \\ &\swarrow_{x'=x, t'=x+x} 1 \times 2 \longrightarrow c', (x + x)[c'/x] \longrightarrow c \\ &\swarrow_{c'=2}^* (x + x)[2/x] \longrightarrow c \\ &\quad = 2 + 2 \longrightarrow c \\ &\swarrow_{c=4}^* \square \end{aligned}$$



We have seen HOFL and a number of different concepts:

- ▶ Type system: type checking and inference (Church vs Curry style)
- ▶ Big-step operational semantics: canonical forms
- ▶ Termination (✗), determinacy (✓), subject reduction (✓), congruence (✗)
- ▶ Two evaluation strategies, leading to lazy and eager semantics
- ▶ We didn't cover: denotational semantics, which requires domain theory.

Next lectures: Concurrency in CCS.



The End